

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №29

Тема: Динамічне програмування

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Опанувати метод динамічного програмування: поняття перекривних підзадач і оптимальної підструктури, підходи «зверху вниз» (memoізація) та «знизу вгору» (табуляція); навчитися будувати таблиці станів, виводити рекурентні співвідношення та відновлювати розв'язок для класичних задач — рюкзак, найдовша спільна підпоследовність, розмін монет, відстань редагування.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal).

Короткі теоретичні відомості

1. Ідея методу. Динамічне програмування (ДП) — спосіб розв'язання задач шляхом розбиття на простіші підзадачі, **розв'язок кожної з яких обчислюється лише один раз** і зберігається для повторного використання. Метод застосовний, коли задача має дві властивості:

- **перекривні підзадачі** — одні й ті самі підзадачі зустрічаються багато разів (як $\text{fib}(3)$ у дереві рекурсії $\text{fib}(6)$, практична робота №16);
- **оптимальна підструктура** — оптимальний розв'язок задачі складається з оптимальних розв'язків її підзадач.

2. Два підходи. Зверху вниз (memoізація): рекурсивна функція, що запам'ятовує результати у словнику/масиві (`functools.lru_cache`); обчислює лише потрібні підзадачі. **Знизу вгору (табуляція):** таблиця заповнюється ітеративно від найпростіших підзадач до задачі повного розміру; немає накладних витрат на рекурсію та ризику переповнення стека, але обчислюються всі підзадачі.

3. Схема розроблення розв'язку ДП:

1. Визначити **стан** — які параметри однозначно задають підзадачу ($\text{dp}[i]$, $\text{dp}[i][j]$).
2. Записати **рекурентне співвідношення** — як dp виражається через менші підзадачі.
3. Задати **базові значення** (початкові умови).
4. Визначити **порядок обчислення** і розмір таблиці; знайти, у якій клітинці міститься відповідь.
5. За потреби **відновити розв'язок** (а не лише його значення) — рухом по таблиці у зворотному напрямку.

4. Класичні задачі та рекурентності:

Задача	Стан	Рекурентне співвідношення
Рюкзак 0/1 (вага w , цінність v , місткість W)	$\text{dp}[i][c]$ — макс. цінність перших i предметів при місткості c	$\text{dp}[i][c] = \max(\text{dp}[i-1][c], \text{dp}[i-1][c - w_i] + v_i)$, якщо $w_i \leq c$
Найдовша спільна	$\text{dp}[i][j]$ — довжина LCS	$\text{dp}[i][j] = \text{dp}[i-1][j-1] + 1$, якщо $X_i = Y_j$;

Задача	Стан	Рекурентне співвідношення
підпоследовність (LCS) рядків X, Y	префіксів $X[:i]$, $Y[:j]$	інакше $\max(dp[i-1][j], dp[i][j-1])$
Розмін суми монетами (мінімум монет)	$dp[s]$ — мінімум монет для суми s	$dp[s] = 1 + \min(dp[s - c])$ за всіма монетами $c \leq s$; $dp[0] = 0$
Відстань Левенштейна $X \rightarrow Y$	$dp[i][j]$ — мін. кількість операцій для $X[:i] \rightarrow Y[:j]$	$dp[i][j] = dp[i-1][j-1]$, якщо $X_i = Y_j$; інакше $1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$

5. Відмінність від жадібного підходу й повного перебору. Повний перебір розглядає всі варіанти (експоненційна кількість), ДП — лише всі різні підзадачі (зазвичай поліноміальна кількість), тобто «перебір із запам'ятовуванням». Жадібний алгоритм (практична робота №30) зберігає лише один варіант на кожному кроці, що швидше, але коректно не для всіх задач.

6. Складність. Для рюкзака $O(n \cdot W)$ за часом і пам'яттю (псевдополіноміальна), для LCS і відстані Левенштейна $O(n \cdot m)$, для розміну монет $O(S \cdot k)$ (k — кількість номіналів). Якщо потрібне лише значення, пам'ять часто зменшують до одного-двох рядків таблиці.

Хід роботи

Завдання 1. Ручне заповнення таблиці ДП: задача про рюкзак

Дано рюкзак місткістю $W = 8$ і предмети (вага; цінність): 1) (2; 3), 2) (3; 4), 3) (4; 5), 4) (5; 8). Знайти найбільшу сумарну цінність і набір предметів.

Порядок виконання:

1. Складіть таблицю dp розміром 5×9 (рядки $i = 0 \dots 4$ — кількість розглянутих предметів, стовпці $c = 0 \dots 8$ — місткість); заповніть рядок $i = 0$ нулями.
2. Заповнюйте рядки по черзі за формулою $dp[i][c] = \max(dp[i-1][c], dp[i-1][c - w_i] + v_i)$; для кожної клітинки запишіть, який варіант (не брати / брати предмет) обрано.
3. Запишіть відповідь $dp[4][8]$ — найбільшу цінність.
4. Відновіть набір предметів: починаючи з клітинки (4; 8), якщо $dp[i][c] \neq dp[i-1][c]$, предмет i взято — зменшіть c на w_i ; рухайтесь до $i = 0$.
5. Перевірте результат повним перебором усіх $2^4 = 16$ підмножин предметів і порівняйте кількість операцій (16 підмножин проти $4 \cdot 9 = 36$ клітинок; поясніть, коли ДП стає значно вигіднішим).

Завдання 2. Ручне заповнення таблиці ДП: найдовша спільна підпоследовність

Знайти довжину та саму найдовшу спільну підпоследовність рядків $X = \text{«ABCBDAB»}$ та $Y = \text{«BDCABA»}$.

Порядок виконання:

1. Складіть таблицю dp розміром 8×7 (рядки — префікси X , стовпці — префікси Y); нульові рядок і стовпець заповніть нулями.
2. Заповніть таблицю за рекурентним співвідношенням для LCS, виділяючи клітинки, у яких символи збіглися (діагональний перехід + 1).
3. Запишіть довжину LCS — значення в правому нижньому куті таблиці.
4. Відновіть одну з найдовших підпоследовностей рухом із правого нижнього кута: при збігу символів додайте символ і переходьте по діагоналі, інакше — до більшого з сусідів (вгору / ліворуч).

5. Поясніть, чому відповідь не обов'язково єдина (знайдіть другу підпоследовність тієї ж довжини).

Завдання 3. Програмна реалізація: рюкзак, LCS, розмін монет

Реалізувати мовою Python алгоритми динамічного програмування з відновленням розв'язку і перевірити їх за ручними результатами.

Порядок виконання:

1. Напишіть функцію `knapsack(items, W)`, що будує таблицю `dp` за формулою з теорії та відновлює список номерів вибраних предметів; поверніть пару (максимальна цінність, номери предметів).
2. Напишіть функцію `lcs(x, y)`, що будує таблицю та повертає рядок — одну з найдовших спільних підпоследовностей.
3. Напишіть функцію `min_coins(coins, amount)`, що знаходить мінімальну кількість монет для суми (`dp[0] = 0`, для недосяжних сум — нескінченність; якщо суму набрати неможливо, поверніть `None`) і набір монет.
4. Перевірте функції на даних із завдань 1, 2 та на розміні суми 6 монетами (1, 3, 4) і суми 7 монетами (2, 4).
5. Для порівняння реалізуйте розв'язок задачі про рюкзак повним перебором підмножин (`itertools.combinations`) і переконайтесь, що ДП та перебір дають однакову цінність для кількох випадкових тестів.

Приклад реалізації мовою Python:

python

```
import random
from itertools import combinations

def knapsack(items, W):
    n = len(items)
    dp = [[0] * (W + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        w, v = items[i - 1]
        for c in range(W + 1):
            dp[i][c] = dp[i - 1][c]
            if w <= c and dp[i - 1][c - w] + v > dp[i][c]:
                dp[i][c] = dp[i - 1][c - w] + v
    chosen, c = [], W
    for i in range(n, 0, -1):
        if dp[i][c] != dp[i - 1][c]:
            chosen.append(i)
            c -= items[i - 1][0]
    return dp[n][W], sorted(chosen)

def lcs(x, y):
    n, m = len(x), len(y)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if x[i - 1] == y[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    result, i, j = [], n, m
    while i > 0 and j > 0:
        if x[i - 1] == y[j - 1]:
            result.append(x[i - 1])
            i -= 1
            j -= 1
        elif dp[i - 1][j] >= dp[i][j - 1]:
            i -= 1
        else:
```

```

        j -= 1
    return "".join(reversed(result))

def min_coins(coins, amount):
    INF = float("inf")
    dp = [0] + [INF] * amount
    last = [0] * (amount + 1)
    for s in range(1, amount + 1):
        for c in coins:
            if c <= s and dp[s - c] + 1 < dp[s]:
                dp[s] = dp[s - c] + 1
                last[s] = c
    if dp[amount] == INF:
        return None
    used, s = [], amount
    while s > 0:
        used.append(last[s])
        s -= last[s]
    return dp[amount], sorted(used)

def knapsack_brute(items, W):
    best = 0
    for r in range(len(items) + 1):
        for subset in combinations(items, r):
            if sum(w for w, v in subset) <= W:
                best = max(best, sum(v for w, v in subset))
    return best

items = [(2, 3), (3, 4), (4, 5), (5, 8)]
print("Рюкзак W = 8:", knapsack(items, 8))
x, y = "ABCBDAB", "BDCABA"
res = lcs(x, y)
print(f"LCS({x}, {y}) = {res}, довжина {len(res)}")
print("Розмін 6 монетами (1, 3, 4):", min_coins([1, 3, 4], 6))
print("Розмін 7 монетами (2, 4):", min_coins([2, 4], 7))

random.seed(3)
same = 0
for _ in range(200):
    its = [(random.randint(1, 10), random.randint(1, 20)) for _ in
range(random.randint(1, 8))]
    cap = random.randint(5, 30)
    same += knapsack(its, cap)[0] == knapsack_brute(its, cap)
print("ДП збігається з повним перебором у", same, "із 200 тестів")

```

Результат виконання програми:

```

Рюкзак W = 8: (12, [2, 4])
LCS(ABCBDAB, BDCABA) = BCBA, довжина 4
Розмін 6 монетами (1, 3, 4): (2, [3, 3])
Розмін 7 монетами (2, 4): None
ДП збігається з повним перебором у 200 із 200 тестів

```

Висновок за результатами: ДП повертає найбільшу цінність 12 для предметів №2 і №4 ($3 + 5 = 8$ за вагою, цінність $4 + 8$), що збігається з ручним розв'язком за таблицею; для LCS знайдено підпоследовність довжини 4 — так само, як у ручній таблиці (відновлена підпоследовність може відрізнитись від іншої, також оптимальної). Розмін суми 6 монетами (1, 3, 4) дає дві монети ($3 + 3$), хоча жадібний вибір («найбільша монета першою») дав би $4 + 1 + 1$ — три монети; суму 7 монетами (2, 4) набрати неможливо. Збіг ДП із повним перебором у всіх 200 випадкових тестах підтверджує коректність рекурентного співвідношення.

Завдання 4. Відстань Левенштейна та порівняння підходів

Реалізувати обчислення відстані редагування між рядками двома підходами (memoізація й табуляція) та порівняти їх ефективність із повною рекурсією.

Порядок виконання:

1. Складіть вручну таблицю dp для слів «sunday» → «saturday» (розмір 7 × 9; перший рядок і перший стовпець – відстані до порожнього рядка); за таблицею визначте мінімальну кількість операцій (вставка, видалення, заміна).
2. Напишіть функцію edit_distance_tab(a, b) — табуляція (двовимірна таблиця) і функцію edit_distance_memo(a, b) — рекурсія з `functools.lru_cache`; перевірте, що результати збігаються на кількох парах слів.
3. Напишіть функцію edit_distance_rec(a, b) — повну рекурсію без запам'ятовування — та підрахуйте кількість рекурсивних викликів для слів «sunday» і «saturday».
4. Порівняйте кількість викликів повної рекурсії з кількістю клітинок таблиці $(\text{len}(a)+1) \cdot (\text{len}(b)+1)$ і виміряйте час для слів довжиною 8–10 символів.
5. Реалізуйте варіант tab-версії з двома рядками пам'яті й переконайтесь, що результат не змінився; поясніть, чому відновити послідовність операцій у цьому варіанті неможливо.

Приклад реалізації мовою Python:

python

```
import time
from functools import lru_cache

def edit_distance_tab(a, b):
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        dp[i][0] = i
    for j in range(m + 1):
        dp[0][j] = j
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1])
    return dp[n][m]

def edit_distance_memo(a, b):
    @lru_cache(maxsize=None)
    def go(i, j):
        if i == 0:
            return j
        if j == 0:
            return i
        if a[i - 1] == b[j - 1]:
            return go(i - 1, j - 1)
        return 1 + min(go(i - 1, j), go(i, j - 1), go(i - 1, j - 1))
    return go(len(a), len(b))

calls = [0]

def edit_distance_rec(a, b):
    calls[0] += 1
```

```

    if not a:
        return len(b)
    if not b:
        return len(a)
    if a[-1] == b[-1]:
        return edit_distance_rec(a[:-1], b[:-1])
    return 1 + min(edit_distance_rec(a[:-1], b), edit_distance_rec(a, b[:-1]),
edit_distance_rec(a[:-1], b[:-1]))

def edit_distance_two_rows(a, b):
    prev = list(range(len(b) + 1))
    for i in range(1, len(a) + 1):
        cur = [i] + [0] * len(b)
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]:
                cur[j] = prev[j - 1]
            else:
                cur[j] = 1 + min(prev[j], cur[j - 1], prev[j - 1])
        prev = cur
    return prev[-1]

pairs = [("sunday", "saturday"), ("kitten", "sitting"), ("abc", "abc"), ("", "abc"),
("algorithm", "altruistic")]
for a, b in pairs:
    t, m, r = edit_distance_tab(a, b), edit_distance_memo(a, b),
edit_distance_two_rows(a, b)
    print(f"{a!r:>12} -> {b!r:<14} відстань: {t} {m} {r} збіг: {t == m == r}")

calls[0] = 0
edit_distance_rec("sunday", "saturday")
print("\nПовна рекурсія для sunday -> saturday: викликів", calls[0], "; клітинок
таблиці:", 7 * 9)

for a, b in (("abcdefgh", "hgfedcba"), ("algorithm", "altruistic")):
    calls[0] = 0
    t0 = time.perf_counter()
    r1 = edit_distance_rec(a, b)
    t_rec = time.perf_counter() - t0
    t0 = time.perf_counter()
    r2 = edit_distance_tab(a, b)
    t_tab = time.perf_counter() - t0
    print(f"{a} -> {b}: відстань {r1}={r2}; рекурсія {calls[0]} викликів, {t_rec:.4f}
с; "
        f"таблиця {(len(a) + 1) * (len(b) + 1)} клітинок, {t_tab:.6f} с")

```

Результат виконання програми:

```

'sunday' -> 'saturday'    відстань: 3 3 3    збіг: True
'kitten' -> 'sitting'     відстань: 3 3 3    збіг: True
'abc' -> 'abc'            відстань: 0 0 0    збіг: True
'' -> 'abc'              відстань: 3 3 3    збіг: True
'algorithm' -> 'altruistic' відстань: 6 6 6    збіг: True

```

```

Повна рекурсія для sunday -> saturday: викликів 153 ; клітинок таблиці: 63
abcdefgh -> hgfedcba: відстань 8=8; рекурсія 123601 викликів, 0.0241 с; таблиця 81
клітинок, 0.000027 с
algorithm -> altruistic: відстань 6=6; рекурсія 744538 викликів, 0.1425 с; таблиця 110
клітинок, 0.000034 с

```

Висновок за результатами: для слів «sunday» → «saturday» відстань Левенштейна дорівнює 3 (вставки «a», «t» та заміна «n» на «r»), що збігається з ручною таблицею. Обидва підходи ДП і двохрядкова версія дають однакові результати, але повна рекурсія без запам'ятовування робить 153 виклики проти 63 клітинок

таблиці для слів «sunday» і «saturday», а для слів із 8–10 літер – вже понад 10^5 викликів (744 538 для «algorithm» → «altruistic») проти ≈ 100 клітинок: перекривні підзадачі обчислюються один раз. Версія з двома рядками зменшує пам'ять з $O(n \cdot m)$ до $O(m)$, але втрачає повну таблицю, необхідну для відновлення послідовності редагувань.

Контрольні питання

1. У чому суть динамічного програмування? Які дві властивості задачі роблять його застосовним?
2. Чим відрізняються підходи «зверху вниз» (memoізація) і «знизу вгору» (табуляція)? Які переваги й недоліки кожного?
3. Запишіть рекурентне співвідношення для задачі про рюкзак 0/1 і поясніть його зміст.
4. Як за заповненою таблицею відновити сам розв'язок (набір предметів, підпослідовність)?
5. Опишіть задачу про найдовшу спільну підпослідовність та її рекурентність.
6. Чому жадібний алгоритм для розміну суми 6 монетами (1, 3, 4) дає неоптимальний результат, а ДП — оптимальний?
7. Яка складність алгоритмів для рюкзака, LCS, відстані Левенштейна? Чому складність рюкзака називають псевдополіноміальною?
8. Як зменшити витрати пам'яті в задачах ДП і яку ціну за це доводиться платити?

Зміст звіту

1. Тема, мета роботи.
2. Заповнені таблиці dp для рюкзака та LCS з відновленням розв'язку (завдання 1, 2).
3. Тексти програм та результати їх роботи (завдання 3, 4).
4. Висновки.